

1. What ShopImpact does

Project purpose, user journey and how the files work together

ShopImpact is a Python/Streamlit shopping journal with separate accounts and permanent saving. Users enter a product type, INR price, brand/shop and date. The app calculates a rough spending-based CO2e estimate, shows monthly progress and suggests reuse/repair options. It implements the conscious-shopping scenario of the Year 1 Python assessment.

The user journey

Step	Result
Create / sign in	A unique email and username create an empty account. Signup immediately opens home and offers a recovery code. Later sign-in accepts email or username plus password.
Add and reflect	Live estimate preview, confirmed purchase saving, month/year totals, category chart, spending/CO2e goals, badges, Turtle leaf and rotating tips.
Explore and review	Category alternatives and comparison; six-month Insights; search/month/category filters; date/price/impact sorting; editing and confirmed deletion.
Keep control	Filtered/full CSV reports, JSON backup/restore, saved contrast preferences, reload after conflicts, sign-out and password-confirmed account deletion.

Execution path

Browser controls → app.py / account_ui.py → logic.py / insights.py / storage.py → accounts.py → SQLite or Neon PostgreSQL → confirmed save and Streamlit rerun.

Streamlit reruns the script after interactions. session_state retains the active token, widgets and account snapshot within that browser session; the database retains purchases/settings after restart. Refresh currently requires another sign-in. Neon hosts PostgreSQL storage; ShopImpact's own code implements account login.

This guide's ten-page map

Page	Topic
2	Data sources, factors, purchase fields and badges
3	app.py, HTML/CSS and visual behaviour
4	Calculation, Insights and backup functions
5	Database and account methods
6	Login/recovery interface and optional email delivery
7	Setup tools and genuine Turtle drawing
8	All test areas and helper code
9	Configuration, libraries, security and Python patterns
10	Worked lifecycle, references and current limitations

Scope: every app/setup function and main code block is explained concisely; repetitive test code is grouped by purpose. Private credentials, personal records and installed dependency source are excluded. The detailed 24-page guide remains available separately.

2. Data and calculation model

Bundled reference values; no invented reuse discount or live exchange rate

The app reads JSON: `product_catalog.json` holds eight categories/factors/alternatives; `emission_sources.json` records five EPA rows, currency conversion, URLs and dataset fingerprint; `alternative_links.json` supplies named official options; `eco_tips.json` holds five authored tips. `tests/sample_purchases.json` contains 15 fictional test entries. No sample purchases populate real accounts.

Where the numerical factors come from

EPA Supply Chain GHG Emission Factors v1.3.0 uses US industries and kg CO₂e per 2022 USD. The app selects **with margins** and divides by the recorded World Bank 2022 annual reference of **78.6044905829916 INR/USD**. EPA source catalogue; original URLs are retained in project metadata.

Categories / proxy	NAICS	kg/USD	Stored kg/INR
New / second-hand clothing	315990	0.12	0.001526630338
Leather footwear (broad footwear)	316210	0.282	0.003587581293
New / refurbished electronics (computer)	334111	0.058	0.000737871330
Single-use / reusable household goods (plastic bottle)	326160	0.579	0.007365991379
Repair services	811490	0.113	0.001437576901

```
factor_per_INR = EPA_factor / reference_exchange_rate
estimate = price_INR × factor_per_INR
INR 500 second-hand clothing → 0.763315169000 → 0.76 kg CO2e.
```

One dictionary per purchase; a list for the whole history

id: UUID (preserved on edits); **product_type**: exact category key; **price**: two-decimal INR string; **brand**: trimmed 1-80 character shop text; **purchase_date**: ISO date between 2000 and 2100; **multiplier**: decimal-string factor; **estimated_co2**: two-decimal estimate; **methodology**: `epa-2022-inr-reference-v2`; **source_code**: EPA NAICS code. Decimal strings keep JSON round trips exact.

Badges are fixed rules, not sustainability certification

Eco Saver: at least 3 monthly entries and at most 50 estimated kg CO₂e. **Low Impact Shopper**: at least 3 flagged entries and at least 60% flagged choices. **Repair Champion**: at least one repair-service purchase. The `lower_impact` flag rewards a reuse/repair/reusable category; it does not alter its numerical multiplier.

CO₂e is carbon dioxide equivalent. Broad US sector proxies, a fixed historical exchange rate and current prices without inflation adjustment are not India/product-specific lifecycle measurements. Free items estimate zero in this price model, not zero physical impact. Reused/new pairs share factors; lifetime/material savings are not quantified.

3. Interface and main code blocks

app.py composes the screens; ui.py supplies the shared presentation

app.py - startup and account state

Imports shared rules/services; set_page_config selects wide layout and title. apply_styles runs before require_account gates access. Loaded purchases/settings and the rotating tip index feed the authenticated screen.

Sidebar and account controls

Shows identity/email and optional verification. Email changes confirm the current password and can send notices. Sign-out revokes the token; Reload drops cached data/widgets. The goals form saves budget, carbon_goal and contrast together and refreshes the settings reference after success.

Header and earlier entries

Signup success opens a home recovery-code panel with download/dismiss actions. An earlier in-memory list can be explicitly rebuilt with current factors and merged by unique IDs. It is not automatic migration of SQLite accounts to Neon.

Dashboard period and entry

Unique recorded/current YYYY-MM values populate month selection. month_records/summary drive spending, estimated CO2e and flagged-choice cards. Four controls collect the purchase; impact updates the preview. The button validates the 1,000-record limit, makes the dictionary and saves at the expected revision. Only success sets feedback and the new dashboard month, then reruns.

Chart, goals and rewards

A radio selects category spending or estimated impact; a loop groups totals for the green bar chart. Goal bars cap at 100% while text keeps actual use/overage. badges supplies labels; flagged choices display the generated leaf. Tips rotate by index modulo the JSON list length.

History view and mutation

Search/month/category/sort feed filter_purchases. Visible results show totals, a table and filtered CSV. The edit selector uses only those records. make_purchase recalculates edits with the same ID; update_purchase replaces the entry. Deletion needs confirmation. No matches hide edit/delete, preserving stored records.

Insights and comparison

Insights uses six logged months and current saved goals: averages, goal balance, previous-calendar-month change, largest category, line chart and optional number table. No prior entries are labelled explicitly. Category suggestions and official links accompany a two-choice price/factor comparison with visible limitations.

Backups, explanation and deletion

Full JSON/CSV exports ignore view filters. Restore requires a file and replacement confirmation; legacy methods need explicit consent. import_json validates/recalculates before saving. How it works explains factors/privacy and offers password-confirmed account deletion. AccountError/SQL errors become normal messages rather than an uncaught crash.

ui.py

apply_styles

Applies scoped CSS with consistent colour variables, responsive headings/cards, readable captions, labelled native controls, focus outlines and a 760px mobile media query.

brand_panel

Draws the authored HTML wordmark/introduction and decorative SVG leaf. It inserts no user text; this leaf differs from the Turtle reward.

What the CSS changes

Scoped .stApp variables coordinate cream/white surfaces, green headings and dark caption text. clamp sizes and a 1,380px maximum width control hierarchy. Rounded metric/form cards, borders, spacing, tabs and white primary-button text create consistency. Caption opacity stays 1; visible focus outlines support navigation.

At 760px the media query changes padding, stacked-row gaps and card/form sizes, and hides secondary brand detail. Double braces escape CSS inside an f-string. unsafe_allow_html is limited to authored style/brand markup, not raw shop-name HTML.

The decorative sign-in SVG differs from the Python Turtle reward. Native labels/controls remain. data-testid selectors may need review after Streamlit upgrades; viewport/contrast checks are not full accessibility certification.

4. Calculation and file functions

Pure Python rules separate the business logic from Streamlit

logic.py

money

Converts to Decimal, rejects invalid/non-finite or out-of-range prices, and rounds to cents with ROUND_HALF_UP. The valid range is INR 0-10,000,000.

impact

Validates the category and price, multiplies by its recorded Decimal factor, then rounds the estimate to 0.01 kg CO2e.

make_purchase

Checks shop text and date, creates/preserves an ID, calculates impact and returns the purchase dictionary. Decimal values become strings for exact JSON storage.

month_records

Filters the list by the first seven ISO-date characters (YYYY-MM), keeping years distinct without changing the original list.

summary

Sums price and estimates with Decimal, counts entries and flagged reuse/repair choices, and returns spend/co2/count/lower; an empty list gives zeros.

badges

Applies the three fixed monthly rules shown on page 2. Short-circuit conditions avoid dividing by zero; goals do not change badge thresholds.

update_purchase

Verifies that the replacement ID exists and returns a new list with only that purchase replaced; a missing ID raises ValueError.

demo_records

Generates 15 fictional varied entries across this and the previous month, including January handling. It is a test fixture, never live-account preload.

insights.py

shift_month

Turns YYYY-MM into a month index, offsets it and uses divmod to recover year/month, correctly crossing December/January.

monthly_trend

Builds six consecutive months ending at the selected period and merges each period with its summary; empty months remain visible.

month_insights

Returns average purchase, goal balance, previous-calendar-month change and largest spending category. Average rounds to cents; missing baselines return None, not invented change.

filter_purchases

Combines case-insensitive search, optional month/category and sorting. Price/impact sorts use Decimal; a new list protects saved data from view changes.

storage.py

export_json

Writes method version, INR currency and purchases as indented JSON. Credentials, sessions and saved goals are excluded.

import_json

Checks size, version, currency, fields and unique IDs; allows at most 1,000 records. Rebuilds every row through make_purchase, rejecting tampered derived values.

export_csv

Writes selected fields into StringIO with DictWriter. An apostrophe protects text beginning with =, +, - or @ from spreadsheet formula interpretation.

Validation before restoration

Backups must use INR and the current method, contain a list with at most 1,000 unique records and stay within 2,000,000 bytes. A legacy checkbox permits explicit recalculation. Rebuilt records prevent uploaded multipliers or estimates from becoming trusted calculations.

ROOT resolves the project directory; CATALOG loads product_catalog.json at import; VERSION identifies the method. Float conversion is used for plotting, not the core Decimal arithmetic. JSON backups carry purchases, not goal settings.

5. Accounts and permanent saving

accounts.py uses parameterised SQL, hashed credentials and revision checks

accounts.py

AccountError

Expected account errors inherit ValueError, so the interface can show normal messages rather than uncaught tracebacks.

ConflictError

An AccountError subtype signals that another tab has a newer saved revision; reload instead of overwriting it.

digest

SHA-256 hashes unpredictable session/recovery/email tokens for lookup. Guessable user passwords use slower Argon2id instead.

username_key

Trims/lowercases usernames; a regex allows 3-40 letters, digits, dots, underscores or hyphens.

check_password

Requires a string of 12-128 characters. It checks length, not compromised-password lists or complex character rules.

email_key

Normalises and syntax-checks ASCII email, length and dots. This does not establish mailbox ownership.

Accounts

The storage service owns a SQLAlchemy engine and authenticated account methods. Purchases/preferences live as JSON text in one user row.

Accounts.__init__

Chooses SQLite or PostgreSQL/psycopg, rejects local storage in public mode, creates/migrates five tables and a unique email index; connections use pre-ping.

Accounts.register

Validates identity/password, creates random IDs and recovery code, stores Argon2/password and recovery digests, then inserts an empty account. Duplicates become friendly errors.

Accounts._identity

Looks up email or username with bound parameters and returns the canonical username, letting both aliases share one login-failure counter.

Accounts._blocked

Checks blocked_until and refuses attempts while the 15-minute lockout remains active.

Five tables: users (identity/hashes, purchases/settings JSON, revision); sessions (token/user/expiry); attempts (failures/blocked time); email_actions (token, purpose, email/password snapshot, expiry); email_requests (hashed address/resend time).

accounts.py

Accounts._failure

Atomically increments failures, restarts expired counters and sets a 900-second lockout after five failures, even for parallel attempts.

Accounts.login

Verifies password (dummy hash for unknown identities), optionally checks verification, then issues a random 12-hour session only if the password snapshot still matches.

Accounts._user

Joins a token digest to a non-expired session/user. A missing or expired match requires another sign-in.

Accounts.load

Decodes the authenticated user's JSON and returns identity, email flag, purchases, settings and revision; password/recovery hashes stay out of the UI payload.

Accounts.save

Authenticates and updates only the expected user/revision, with a 1,000-entry limit. Success increments revision; stale writes raise ConflictError.

Accounts.logout

Deletes the supplied token's session row, revoking that login; the UI clears browser state separately.

Accounts.reset_password

Checks recovery with compare_digest and its own lockout, conditionally rotates password/recovery once, and revokes sessions and email actions.

Accounts.set_email

Confirms the current password, changes to a unique normalised address, resets its verified flag and invalidates outstanding email links.

Accounts.request_email_action

Applies a 60-second hashed-address cooldown; silently skips unknown/ineligible users. Stores a purpose/email/password-bound token digest with a 24-hour verify or 30-minute reset expiry.

Accounts.consume_email_action

DELETE RETURNING atomically claims an unexpired token once. Matching account details permit verification or password/recovery rotation; reset revokes sessions and clears relevant lockouts.

Accounts.delete_account

Requires session and password, then transactionally deletes the user, sessions and email actions. User-row deletion removes purchases/settings; downloaded backups remain separate.

6. Login, recovery and optional email

Immediate signup-to-home; Gmail sends mail, not Google OAuth sign-in

account_ui.py

_cached_account_service

@st.cache_resource reuses the Accounts instance/engine for its configuration and implementation identity across reruns.

account_service

Includes id(Accounts) in the cache key, preventing stale account/exception implementations after code reload. Its clear alias supports isolated tests.

email_settings

Gathers eight private email keys from environment, otherwise st.secrets, and tolerates a missing secret file. Actual values are not documented here.

configured_mailer

Returns None when sending is disabled; otherwise builds a validated Mailer. Recovery-code access works without Gmail.

email_link_screen

Moves checked link parameters into session state and clears the URL. Confirmation consumes a verify/reset token; reset clears sessions and exposes the replacement recovery code.

request_email_form

Renders a purpose-specific form, disabled without a mailer. Valid requests queue asynchronously and show the same generic account-eligibility response.

sign_out

Revokes the database session, clears session_state and reruns into the account gate.

require_account

Loads configuration and handles email links. It validates an existing session or renders Sign in/Create/Recover and stops. Signup immediately logs in and reruns to home.

save_account

Saves at the expected revision and catches account/database errors. It updates the in-memory account only after confirmed success and returns a boolean.

Current verification policy

Sign in, Create account and Recover account are the three entry tabs. There is no Verify email tab. require_account passes verification=False: signup/home access is immediate. Optional sidebar verification is needed for email-reset eligibility, not ordinary login.

EMAIL_REQUEST_MESSAGE is generic. Streamlit can copy secrets into environment variables; isolated previews use a separate secrets file.

email_delivery.py

EmailConfigurationError

Marks invalid provider, sender, link or transport settings as configuration errors.

Mailer

A dataclass holds SMTP and app-URL settings; repr=False prevents ordinary object representation from exposing settings/password.

Mailer.__post_init__

Checks sender, host, required credentials, port and SSL/STARTTLS. Public URLs need HTTPS; only localhost may use HTTP. URL credentials/query/fragments are rejected.

Mailer.connect

Creates a certificate-validating SSL context and encrypted SMTP connection. STARTTLS precedes login; a 15-second timeout and cleanup bound failures.

Mailer.check_login

Opens/closes a secure authenticated connection without sending mail, supporting transport checks.

Mailer.send_message

Validates recipient, builds a plain EmailMessage and submits via the secure provider connection. Acceptance does not prove inbox delivery.

Mailer.send_action

URL-encodes purpose/token in a trusted app link, selects verify/reset subject and expiry, and sends single-use instructions.

mailer_from_settings

Builds Mailer from private settings, defaulting to smtp.gmail.com, port 465 and SSL; sender/password/app URL still need configuration.

queue_email_action

Acquires one of eight slots without waiting and submits a task to two workers; failed submission releases the slot.

queue_email_action.deliver

Requests an eligible account token and sends it; suppresses private provider errors and always releases its slot. The queue has no durable retries.

queue_notice

Queues a plain account-change message only when an address and slot are available.

queue_notice.deliver

Sends that notice, suppresses failures and releases the semaphore in finally; it creates no action token.

7. Setup tools and Turtle artwork

Local owner setup is separate from the public shopping interface

tools/connect_database.py

validate_url

Requires complete PostgreSQL credentials/host/database and sslmode=require or stronger, selects psycopg and preserves provider query parameters.

read_clipboard

On Windows, reads copied text through hidden PowerShell with a timeout; credentials are not placed in arguments or printed.

main

Accepts hidden input or --clipboard, validates it and calls configuration; safe return codes distinguish success/failure.

connection_error_message

Translates authentication, DNS, timeout, refusal and TLS failures to fixed explanations without returning raw provider text.

configure_database

Tests SELECT 1, initialises the public-mode schema and disposes engines. Only then atomically merges/replaces private TOML settings; failures preserve previous settings.

tools/database_setup_ui.py

Module-level Streamlit code adds the project path, loads shared setup helpers and checks loopback server.address. A masked URI form validates input; a spinner captures credential-free helper output. Success links to the app; failure shows only a sanitised message. No separate function is defined.

tools/email_setup_ui.py

Another loopback-only module form collects sender, masked Google app password and app URL. It removes password spaces, validates email, sends a test to the sender through SSL port 465, then atomically merges private TOML settings. Failure removes temporary files and preserves old settings. SMTP acceptance is not guaranteed inbox delivery.

The Gmail setup page has a stale string suggesting verification becomes required. Actual require_account code keeps it optional for home access. Setup never automatically transfers local SQLite accounts to Neon; export/restore purchases when switching.

tools/draw_rewards.py

SvgTurtle

Subclasses the standard turtle.TNavigator so real forward/turn geometry can be recorded as SVG without a Tk window.

SvgTurtle.__init__

Initialises navigation, pen state, buffered paths, colour and line width.

SvgTurtle.goto

Records start/end points while drawing, then calls inherited navigation; pen-up movements leave no stroke.

SvgTurtle.flush

Saves a non-empty buffered path with its colour/width and resets the buffer.

SvgTurtle.penup

Flushes the stroke and disables drawing before repositioning.

SvgTurtle.pendown

Enables stroke recording for subsequent movement.

SvgTurtle.pencolor

Flushes earlier geometry, then changes the colour for later strokes.

SvgTurtle.pensize

Flushes earlier geometry, then changes subsequent stroke width.

SvgTurtle.svg

Outputs the SVG title/background/polylines. Negated y coordinates convert Turtle geometry to SVG; rounded joins finish the leaf.

draw_leaf

Nested forward/left loops create two curved sides, then the central vein and smaller veins. One recipe works for SVG and desktop Turtle.

main

Parses --preview, writes assets/turtle/leaf.svg, and optionally opens turtle.Turtle/Tk to draw the same recipe. The direct-run guard invokes it.

The app displays the saved SVG, not live Turtle animation. --preview uses turtle.Turtle and the Tk event loop; a working desktop preview remains unverified in the bundled runtime. This tool imports argparse, Path and standard-library turtle.

8. Tests and verification evidence

78 automated checks; disposable accounts, mocked setup/SMTP and real UI tests

test_logic.py - 20 checks

Decimal formula/rounding; invalid prices, shop text, dates and categories; free entries; month/year grouping and known totals; empty rewards and badge boundaries; edit recalculation/IDs; 15-row January fixture; alternatives/links; JSON round-trip and tampered estimates; explicit legacy migration; source factors/no invented reuse discount; duplicate IDs/malformed uploads; spreadsheet-formula safety.

test_accounts.py - 16 checks

Separate users and restart persistence; salted hashes; duplicate/weak credentials; login lockout; conflicting revisions; recovery rotation/session revocation; reset racing login; logout/expiry; password-confirmed deletion; public-mode SQLite rejection; case-insensitive email login; invalid/duplicate email; password-confirmed email changes; shared alias lockout; recovery by email; additive migration retaining users/purchases/sessions.

test_app.py - 12 checks

Empty dashboard; filtered edit choices without changing saved data; current saved goal reflected in Insights; polite incorrect-login errors; blank-shop rejection; add/edit/delete; saved preferences and refresh persistence; signup followed by email login; linking existing email; removing query tokens and confirmation; reset revoking sessions; email-enabled signup still opening home immediately.

test_account_service.py - 1 check

A reloaded Accounts implementation receives a fresh cached resource, preventing stale exception-class problems during development. Nested ReloadedAccounts simulates that change.

test_database_setup.py - 7 checks

Safe diagnosis for password/DNS/timeout/refusal/TLS errors; clipboard failure privacy; copied input reaching validation; escaped credentials/TLS parameters preserved; insecure/incomplete/non-PostgreSQL URIs rejected; invalid input hidden; connection failure never printing credentials or replacing settings.

test_email_actions.py - 9 checks

Optional stricter verification gate, token hashing and one-use; expired/wrong-purpose rejection; verified-only reset and silent unknown address; cooldown/old-link invalidation; reset revoking sessions and rotating recovery; email-change invalidation; recovery-code reset invalidating links; simultaneous claims allowing one success; deletion removing email actions.

test_email_delivery.py - 6 checks

Reject untrusted/insecure URLs and transport; representation hides credentials; correct purpose/link/expiry text; SSL transport; STARTTLS before login; unknown recipients and worker errors cause no exposed message or unintended send.

test_insights.py - 7 checks

Year boundaries; six months including blanks and exact totals; rounded average/balance/change; missing baseline; free prior purchase as valid zero baseline; combined filters leaving records unchanged; numeric price/impact and date sorting.

Helpers and repeated test structure

unittest.TestCase groups checks; setUp creates fixtures, tearDown disposes engines/patches/temp files. by_label selects UI controls; app launches/signs into AppTest; purchase/mailer build synthetic inputs; allow_request resets cooldown; verify prepares a verified account. reset_after_verification creates a login/reset race; consume tests parallel token claims. Mock/patch replace external calls; assertEquals/True/False/Raises and subTest check expected outcomes.

Recorded verification: all 78 passed on 3 October. Browser checks covered sign-in, signup-to-home, desktop/mobile, contrast, history filters and Turtle rewards. Synthetic screenshots are evidence for tested views, not human feedback or every browser/device.

9. Libraries, configuration and privacy

How to run it and why the Python patterns are used

Pinned libraries and standard Python

Streamlit 1.55.0: controls, forms, charts, reruns and session state. **SQLAlchemy 2.0.43:** database engines, parameters and transactions. **psycopg[binary] 3.2.10:** PostgreSQL driver. **argon2-cffi 25.1.0:** Argon2id password hashing. requirements.txt pins these versions. toml is available with the current Streamlit dependencies for setup files.

Standard-library JSON/CSV handle portable data; datetime validates dates; Decimal keeps exact arithmetic; Path resolves files; uuid/secrets create IDs/tokens; hashlib stores token digests; re checks formats; time handles expiry; ssl/smtplib/email send encrypted messages; threading/concurrent futures bound email work; turtle draws geometry.

Public project configuration

.streamlit/config.toml: green primary colour, cream background, pale secondary surfaces, dark sans-serif text and 2 MB upload limit. **.vscode/tasks.json:** install/run/test tasks use the project virtual environment.

.vscode/launch.json: debugpy runs the Streamlit module in the integrated terminal. **.github/workflows/tests.yml:** future push/PR jobs use Ubuntu/Python 3.12, install requirements and run unittest.

.gitignore: excludes .venv, Python caches, secrets/private temporary files and personal_data. **README/docs:** explain features, running, design, sources, testing, Neon/Gmail setup, deployment, assessment status and genuine-feedback collection. Turtle artwork and synthetic fixtures are shareable.

Run from the SA PYTHON folder

```
.\.venv\Scripts\python.exe
-m streamlit run app.py
```

The executable and arguments above form one terminal command. On another computer, first install Python 3.12, create .venv and install requirements.txt. Tests use -m unittest discover -s tests -v; tools/draw_rewards.py generates the SVG.

Credentials and authenticated saving

Private .streamlit/secrets.toml holds the database URI and optional SMTP settings. Values are excluded from this PDF and Git. Passwords have salted Argon2id hashes; random session/recovery/email tokens have SHA-256 digests. Bound SQL parameters avoid inserting user input into SQL text. Transactions and revision checks prevent stale purchases from silently replacing newer data.

Five failed login/recovery attempts block that identifier for 15 minutes; email requests have a 60-second cooldown. Sessions expire after 12 hours. Recovery/reset rotates codes and revokes old sessions. Changing email invalidates action links. Verification expires after 24 hours; reset after 30 minutes; tokens are claimed atomically once.

Python constructs, once explained

Imports/constants/docstrings: expose libraries, shared settings and non-executable explanation.

Lists/dictionaries/sets: records, keyed fields, unique months/IDs. **Functions/classes:** reusable rules and related service operations. **if/loops/comprehensions:** validate, select and transform records; sum/max/any/sorted summarise them.

try/except/finally: friendly errors and guaranteed slot cleanup. **with:** closes files/connections and commits/rolls back transactions. **Decorators/dataclasses:** cache the service and structure mail settings. **f-strings:** format values/CSS. **__main__:** runs tools/tests directly without executing them on ordinary import.

Practical privacy limits

Each app session loads its own account, but the database administrator can access records. Purchase data is not end-to-end encrypted. Account deletion does not erase downloaded backups. The email queue is in-memory, with no durable retry/delivery monitoring. Gmail SMTP is separate from Google OAuth, which is not implemented.

10. Worked lifecycle and references

A complete purchase example, source trail and honest project status

From new account to a saved result

Step	What the code does
1. Register	account_ui checks confirmation; Accounts.register validates identity, hashes credentials and inserts [] purchases/default settings. Accounts.login returns a token; Streamlit reruns straight to home.
2. Enter INR 500	Select Second-hand clothing, a shop and valid date. impact uses Decimal and the bundled 0.001526630338 factor; preview shows 0.76 estimated kg CO2e.
3. Confirm saving	make_purchase returns the dictionary. save_account calls Accounts.save with token and revision; one conditional update advances revision. Success refreshes the selected month and shows feedback.
4. Reflect	summary adds monthly totals; badges checks its fixed rules; a flagged choice displays the Turtle leaf. Insights compares logged calendar periods and uses current saved goals.
5. Correct or export	Edit keeps the ID but recalculates fields. Filters change the view, not records. CSV protects formula-like text. JSON restoration checks/rebuilds every row before an authenticated replacement save.
6. Sign out / recover	logout deletes the session and clears UI state. Recovery rotates the code and password, invalidates email actions and revokes all existing sessions.

Source trail

Numerical values and mappings come from **data/emission_sources.json** and **data/product_catalog.json**. The EPA catalogue was checked while preparing the detailed guide. The recorded World Bank request is a historical reference, not a live rate. Named options are retained from official pages checked in the app on 2 October; terms/availability can change.

EPA dataset catalogue

World Bank recorded 2022 INR/USD API request

Python Decimal

Python Turtle

Streamlit database guidance

Implemented and still pending

Defaults: budget=10000.0, carbon_goal=100, contrast=False. Argon2id uses time_cost=2, memory_cost=19456, parallelism=1. SQLite BEGIN IMMEDIATE serialises migrations; older email columns are added without replacing purchases.

The local app, multi-user saving with Neon, account recovery, calculations, dashboard, Insights, filters, backups, rewards and polished UI are implemented. All 78 recorded tests passed. Still pending: private Gmail sender setup and real inbox delivery, public Streamlit deployment/redeployment checks, human usability feedback, full accessibility review, desktop Tk preview and the final assessed submission with actual student/repository details.

This guide covers all 20 authored Python files through named app/setup functions, module-level screens and grouped test explanations, plus data/assets/configuration. It is not installed-library source, a verbatim code printout, certified carbon accounting or a guarantee of an assessment grade. Use the longer guide for detailed source-line references.